Kerni: guion de presentación

Una mascota de escritorio que cuida la salud de tu PC con Windows. Reparto entre 6 personas; la demostración va al final y la hace una sola persona.

Orden	Persona	Tema	Tiempo
1	Persona 1	El problema y la propuesta	2 min
2	Persona 2	El personaje y la experiencia	3 min
3	Persona 3	Cómo está construido (arquitectura)	3 min
4	Persona 4	Seguridad: Kerni propone, la persona decide	3 min
5	Persona 5	Memoria y conocimiento (Supabase y RAG)	3 min
6	Persona 6	DEMOSTRACIÓN EN VIVO (la última)	6 min

Total sugerido: 20 minutos más preguntas. Los tiempos son una propuesta: si el tiempo real es otro, se recortan los puntos marcados como opcionales.

Reglas del reparto

- **Una sola voz por tema.** Nadie repite lo del anterior; si hace falta, lo enlaza con una frase.
- **La demostración es lo último y es de una sola persona** (Persona 6). Las demás no tocan la PC de la demo mientras tanto.
- Cada persona termina con su frase de cierre, que sirve de puente al siguiente tema.
- Poner los nombres reales en cada bloque antes de ensayar. Ensayar la secuencia completa al menos una vez, con cronómetro.

En una frase

Kerni es un compañero de escritorio, en el estilo de Bitoo, que mira CPU, memoria, disco y procesos reales de la PC, conversa con la persona en español y le **propone** acciones (cerrar un programa que se pasó, limpiar temporales). Nada se ejecuta sin que la persona lo apruebe en pantalla.

Bloques de la presentación

1 Persona 1: El problema y la propuesta

2 min · abre la presentación

Qué dice

- Las PC se vuelven lentas y la mayoría de la gente no sabe por qué: el Administrador de tareas es técnico y asusta.
- Kerni es el "doctor de la compu": explica en palabras simples qué pasa y qué conviene hacer.
- Alcance del primer hito: **Salud y Pánico** (leer el estado real y reaccionar cuando la PC se ahoga).
- Es hermano de Bitoo: mismo patrón de agente y misma app de escritorio, pero con acciones locales en lugar de pagos.

Qué muestra

- Diapositiva o pantalla con el nombre y la mascota.
- Una imagen de Kerni sonriente y otra asustada, para anticipar lo que se verá.
- Opcional: una frase sobre cómo nació la idea (propuesta CyberPet AI).

Frase de cierre: "Veamos cómo se ve y se siente Kerni."

2 Persona 2: El personaje y la experiencia

3 min

Qué dice

- Kerni vive en una ventana transparente, siempre visible, con icono en la bandeja del sistema.
- Tiene **8 modos** hechos en Rive (base, sonriente, normal, aburrido, asustado, alerta, investigando y dormido) y una animación de saludo.
- La cara refleja el estado: asustado cuando la CPU se sostiene al tope, alerta si propone algo, dormido tras 3 minutos sin uso, y según la **batería** en reposo (70 % o más sonriente, 30 a 69 % normal, menos aburrido).
- En reposo se vuelve semitransparente para no estorbar y las métricas se pueden ocultar.

Qué muestra

- Capturas o el propio Kerni cambiando de cara.
- El saludo animado y las barras de CPU, RAM y disco.
- El botón de ocultar métricas.

Frase de cierre: "Pero lo importante está debajo: ¿cómo funciona?"

3 Persona 3: Cómo está construido

3 min

Qué dice

- **App de escritorio:** Tauri (Rust + interfaz web). Rust es lo único que toca el sistema: lee métricas y ejecuta acciones aprobadas.
- **Backend:** Node 22 con Fastify. Recibe el mensaje y una "foto del sistema", corre el bucle del agente y devuelve respuesta y propuestas.
- **Cerebro intercambiable:** Anthropic, Gemini u OpenRouter, sin cambiar el resto del código.
- **Reglas sin modelo:** el vigilante de RAM y las respuestas de disco funcionan sin llave ni internet.
- Todo se prueba: pruebas automáticas en el backend, el front y Rust, más una prueba contra la ventana real.

Qué muestra

- Un diagrama simple: App (Rust) - foto del sistema - Backend - modelo - propuesta - tarjeta - Rust.
- Opcional: la carpeta del repositorio y el comando de pruebas en verde.

Frase de cierre: "Un agente con poder sobre una PC necesita límites claros."

4 Persona 4: Seguridad: Kerni propone, la persona decide

3 min

Qué dice

- El modelo **nunca ejecuta nada**. Propone con una herramienta, el backend la valida y la app muestra una tarjeta; solo tras pulsar *Aprobar* se actúa.
- **Doble validación:** el backend revisa la propuesta contra la foto del sistema y Rust vuelve a revisarla antes de tocar nada.
- **Procesos protegidos:** los de Windows y seguridad no se pueden cerrar, aunque el modelo lo pida.
- Los nombres de procesos y archivos se tratan como **datos, no instrucciones** (un proceso llamado "ignora tus reglas" no engaña a Kerni).
- Las llaves viven solo en un archivo local ignorado por git. Los commits son de personas del equipo, sin atribuciones de herramientas.

Qué muestra

- La tarjeta de confirmación con el botón Aprobar y Rechazar.
- Opcional: un caso de prueba que rechaza cerrar svchost.

Frase de cierre: "Y además de actuar con cuidado, Kerni puede acordarse de vos."

5 Persona 5: Memoria y conocimiento (Supabase y RAG)

3 min

Qué dice

- **Memoria en Supabase:** Kerni recuerda hechos que la persona aprueba guardar (para qué usa la PC), retoma la conversación y resume la salud de las últimas horas.
- Nada se guarda solo: proponer un recuerdo también pasa por la tarjeta. Se rechazan contraseñas, llaves y rutas con el nombre de usuario.
- Las tablas tienen seguridad por filas sin políticas: las llaves públicas no leen nada; solo el backend, con la llave secreta.
- **RAG con pgvector:** una base de conocimiento sobre PCs, hardware, Windows, Office y buenas prácticas, buscada por significado y citada en la respuesta.
- Si Supabase o el RAG fallan, Kerni sigue funcionando sin ellos.

Qué muestra

- El esquema de tablas y el flujo: pregunta - embedding - fragmentos - respuesta.
- La carpeta de conocimiento con sus documentos.

Frase de cierre: "Ahora lo vemos funcionando en vivo."

6 Persona 6: DEMOSTRACIÓN EN VIVO

6 min · última, a cargo de una sola persona

Qué dice

- Narra lo que hace mientras lo hace, en voz alta y despacio.
- Sigue el guion de la página siguiente; si algo falla, pasa al plan B sin disculparse largo.
- Cierra con una frase corta y da paso a las preguntas.

Qué muestra

- Kerni real en el escritorio (abierto con **Kerni.bat**).
- Ver la página siguiente: secuencia, preparación previa y plan B.

Frase de cierre: "Kerni cuida la PC, pero la decisión siempre es de la persona. Gracias; ¿preguntas?"

Guion de la demostración (Persona 6)

N.º	Qué hacer	Qué se debe ver	Depende de
1	Kerni ya abierto. Dejar que salude y mostrar las barras. Ocultar y volver a mostrar las métricas.	Cara normal o sonriente según batería; barras de CPU, RAM y disco.	Solo la app
2	Escribir: *cuánto disco me queda*.	Respuesta inmediata con el porcentaje de disco, sin usar el modelo.	Solo la app
3	Abrir el navegador con la página de demostración y agregar `?panico` a la dirección.	Kerni asustado y el panel con los procesos sospechosos (datos de demostración).	Servidor web del front
4	Escribir: *mi compu va lenta, qué hago*.	Respuesta breve con datos reales; si propone algo, aparece la tarjeta. **Rechazar o aprobar solo "limpiar temporales"**, nunca cerrar un programa real.	Backend y una llave de modelo
5	Escribir: *cuánta RAM necesito para programar*.	Respuesta apoyada en la base de conocimiento, citando la guía de memoria.	Supabase, RAG indexado y embeddings
6	Escribir: *uso esta compu para programar* y aprobar el recuerdo en la tarjeta.	Tarjeta "¿Querés que recuerde esto?"; al aprobar, Kerni dice que lo anotó.	Supabase y memoria activada

Los pasos 4 a 6 solo se muestran si se probaron el mismo día con la llave y el proyecto de Supabase reales. Lo que no se haya probado en vivo no se promete en la presentación.

Preparación (el día anterior y 30 minutos antes)

- Ejecutar `Kerni.bat check` y después `Kerni.bat`; confirmar que el backend informa el cerebro activo.
- Probar la llave: `npm run probar-llm` en la carpeta del backend. **El plan gratuito de Gemini permite unos 20 pedidos por día por modelo**: no gastarlos ensayando el mismo día.
- Si se usa memoria y RAG: correr las migraciones en Supabase, `npm run probar-memoria` y `npm run indexar`.
- Cargar el portátil y cerrar notificaciones; dejar la ventana de Kerni visible y el navegador con la página de demostración lista.
- Ensayar la secuencia completa con cronómetro, tal como se hará.

Plan B si algo falla

- **Sin internet o sin cupo del modelo:** usar solo los pasos 1 a 3, que funcionan sin modelo, y contar lo demás con el diagrama.
- **Kerni no abre:** mostrar la interfaz en el navegador con datos de demostración (agregar `?panico` para el modo pánico).
- **Respuesta lenta del modelo:** hablar mientras se espera (el globo dice "Pensando") y no repetir el mensaje.
- Tener un video corto de la demo grabado el día anterior como último recurso.

Preguntas probables y cómo responderlas

¿Puede Kerni borrar o cerrar cosas por su cuenta?
No. Solo propone. La persona ve una tarjeta y aprueba; además, backend y Rust validan la acción y los procesos de Windows y seguridad están protegidos.

¿Qué pasa si el modelo se equivoca o alguien intenta engañarlo?
Nada se ejecuta por su palabra: el pid y el nombre se comparan contra la lectura real del sistema. Los nombres de procesos son datos, no instrucciones.

¿Funciona sin internet o sin llave?
Sí en parte: la lectura de métricas, el vigilante de RAM y las respuestas de disco son reglas locales. La conversación y el conocimiento necesitan modelo.

¿Por qué la cara depende de la batería y no de la CPU?
Fue la regla pedida (carga alta, normal, baja). Se corrigió una lectura equivocada inicial y quedó documentada en el registro de prompts del repositorio.

¿Dónde se guardan mis datos?
Los recuerdos, turnos de chat y un resumen de salud van a Supabase, solo con aprobación para los recuerdos, y la persona puede borrarlos. Las llaves nunca salen del equipo.

¿Qué falta?
Probar memoria y RAG contra un proyecto real de Supabase y una llave de embeddings, un panel para ver y borrar recuerdos, icono propio, congelar procesos en el modo pánico y los siguientes módulos de la propuesta.

¿Quién hizo qué y cómo se usó la IA?
Hay un registro de cada sesión con IA en la carpeta de prompts del repositorio (pedido, decisiones y verificación). Los commits están a nombre del equipo, sin atribuciones de herramientas.

> **Antes de presentar**
>
> - Reemplazar "Persona 1" a "Persona 6" por los nombres reales.
> - Ajustar los tiempos al total disponible.
> - No afirmar nada de memoria o RAG que no se haya probado con el proyecto real antes de la presentación.